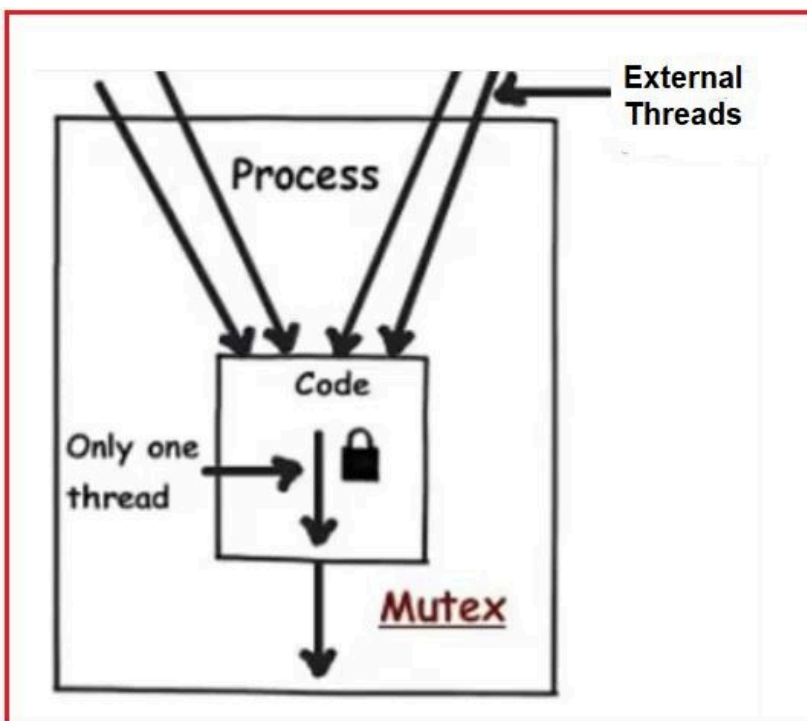


Mutex Class in C# with Example

In this article, I am going to discuss **Mutex Class in C#** i.e. How to use Mutex in C# to Protect Shared Resources in Multithreading with Examples. Please read our previous article where we discussed [How to Protect Shared Resources in Multithreading using Monitor Class in C#](#) with Examples.

Why Mutex as we already have Lock and Monitor for Thread Safety?

Mutex also helps us to ensure that our code is thread-safe. That means when we run our code in a multi-threaded environment then we don't end up with inconsistent results. The Locks and Monitors ensure thread safety for threads that are InProcess i.e. the threads that are generated by the application itself i.e. Internal Threads. But if the threads are coming from OutProcess i.e. from external applications then Locks and Monitors have no control over them. Whereas Mutex ensures thread safety for threads that are Out-Process i.e. the threads that are generated by the external applications i.e. External Threads.

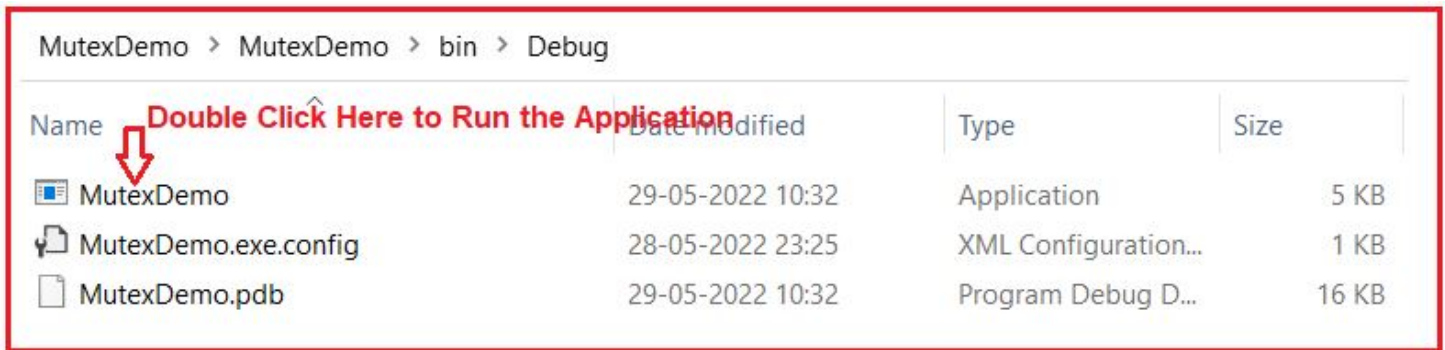


Let us first understand what is External Process Or External Threads and then we will understand Mutex in C#. Let us first create a console application and then copy and paste the below code into it.

```
using System;
namespace MutexDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Application Is Running.....");
            Console.ReadKey();
        }
    }
}
```

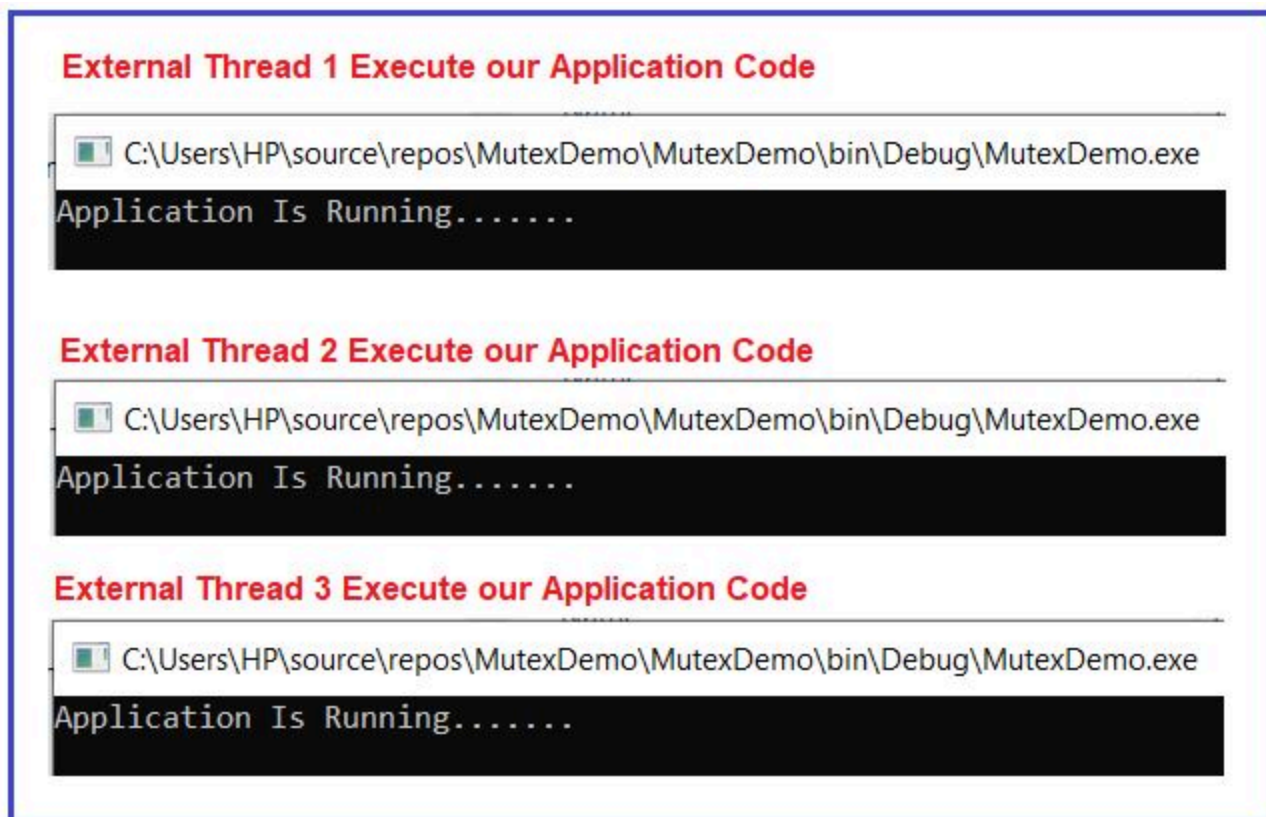
}

Now, build the project and go to the project's bin\Debug directory and there you will find the application EXE file as shown in the below image.



MutexDemo > MutexDemo > bin > Debug			
Name	Date Modified	Type	Size
 MutexDemo	29-05-2022 10:32	Application	5 KB
 MutexDemo.exe.config	28-05-2022 23:25	XML Configuration...	1 KB
 MutexDemo.pdb	29-05-2022 10:32	Program Debug D...	16 KB

When you double-click on the Application EXE file, an external thread will execute the application code. And now if you double-click multiple times, then each time a new external thread will be created and execute our application code as shown in the below image. I double-click on the EXE three times, so three external threads are accessing our application at the same time.



External Thread 1 Execute our Application Code

```
C:\Users\HP\source\repos\MutexDemo\MutexDemo\bin\Debug\MutexDemo.exe
Application Is Running.....
```

External Thread 2 Execute our Application Code

```
C:\Users\HP\source\repos\MutexDemo\MutexDemo\bin\Debug\MutexDemo.exe
Application Is Running.....
```

External Thread 3 Execute our Application Code

```
C:\Users\HP\source\repos\MutexDemo\MutexDemo\bin\Debug\MutexDemo.exe
Application Is Running.....
```

But, if you want to restrict that only external thread access to our application code at any given point in time, then we need to use the Mutex in C#. Now, let us rewrite the previous example using the C# Mutex Class and see what happens when we try to access the application code multiple times from outside using External Threads, and then we will discuss the C# Mutex Class in detail.

Modify the code Program.cs class file as follows. So, when one external thread accesses our code, no more external threads can access the code. The following example exactly does the same using C# Mutex Class.

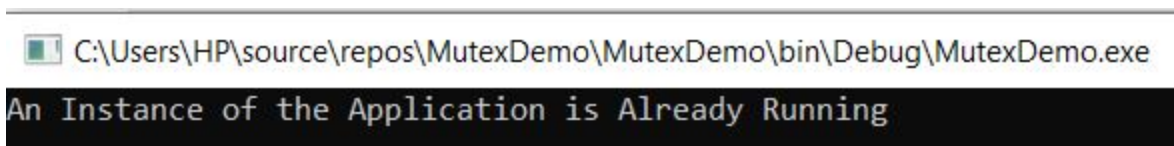
This is one of the use cases of Mutex in C#. Don't go with the code, I will explain the code later part of this article.

```
using System;
using System.Threading;
namespace MutexDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            using(Mutex mutex = new Mutex(false, "MutexDemo"))
            {
                //Checking if Other External Thread is Running
                if(!mutex.WaitOne(5000, false))
                {
                    Console.WriteLine("An Instance of the Application is Already Running");
                    Console.ReadKey();
                    return;
                }
                Console.WriteLine("Application Is Running.....");
                Console.ReadKey();
            }
        }
    }
}
```

Now close all the instances which are running. Then build the project and again go to the projects bin\Debug directory and again click on the Exe file for the first time. You will get the following result.



Now, again click on the EXE file. This time it will wait for 5 seconds and then it will give you the following message. This ensures that only one external thread can access our application code at any given point in time.



Now, I hope you understand the need for Mutex Class in C#. Let us proceed further and try to understand the Mutex Class in C# with multiple examples.

What is Mutex Class in C#?

Mutex works like a lock i.e. acquired an exclusive lock on a shared resource from concurrent access, but it works across multiple processes. As we already discussed exclusive locking is basically used to ensure that at any given point in time, only one thread can enter into the critical section.

When two or more threads need to access a shared resource at the same time, the system needs a synchronization mechanism to ensure that only one thread at a time uses the resource. Mutex is a synchronization mechanism that grants exclusive access to the shared resource to only one external thread. If a thread acquires a mutex, the second thread that wants to acquire that mutex is suspended until the first thread releases the mutex. If this is not clear at the moment then don't worry, we try to understand this with some examples.

Constructors, and Methods of Mutex Class in C#:

Let us understand the different Constructors, and Methods of the Mutex Class in C#. If you right-click on the Mutex class and select go to definition, then you will see the following. As you can see, Mutex is a sealed class and it is inherited from WaitHandle class. As it is a sealed class, so no further inheritance is possible i.e. no class can be derived from this sealed Mutex class in C#.

```
namespace System.Threading
{
    ...public sealed class Mutex : WaitHandle
    {
        ...public Mutex();
        ...public Mutex(bool initiallyOwned);
        ...public Mutex(bool initiallyOwned, string name);
        ...public Mutex(bool initiallyOwned, string name, out bool createdNew);
        ...public Mutex(bool initiallyOwned, string name, out bool createdNew, MutexSecurity mutexSecurity);

        ...public static Mutex OpenExisting(string name);
        ...public static Mutex OpenExisting(string name, MutexRights rights);
        ...public static bool TryOpenExisting(string name, out Mutex result);
        ...public static bool TryOpenExisting(string name, MutexRights rights, out Mutex result);
        ...public MutexSecurity GetAccessControl();
        ...public void ReleaseMutex();
        ...public void SetAccessControl(MutexSecurity mutexSecurity);
    }
}
```

Constructors of Mutex Class in C#:

The Mutex Class in C# provides the following five constructors that we can use to create an instance of the Mutex class.

1. **Mutex()**: It initializes a new instance of the Mutex class with default properties.
2. **Mutex(bool initiallyOwned)**: It initializes a new instance of the Mutex class with a Boolean value that indicates whether the calling thread should have initial ownership of the mutex.
3. **Mutex(bool initiallyOwned, string name)**: It initializes a new instance of the System.Threading.Mutex class with a Boolean value that indicates whether the calling thread should have initial ownership of the mutex, and a string that is the name of the mutex.

4. **Mutex(bool initiallyOwned, string name, out bool createdNew):** It initializes a new instance of the System.Threading.Mutex class with a Boolean value that indicates whether the calling thread should have initial ownership of the mutex, a string that is the name of the mutex, and a Boolean value that, when the method returns, indicates whether the calling thread was granted initial ownership of the mutex.
5. **Mutex(bool initiallyOwned, string name, out bool createdNew, MutexSecurity mutexSecurity):** It initializes a new instance of the System.Threading.Mutex class with a Boolean value that indicates whether the calling thread should have initial ownership of the mutex, a string that is the name of the mutex, and a Boolean variable that, when the method returns, indicates whether the calling thread was granted initial ownership of the mutex, and the access control security to be applied to the named mutex.

Following are the parameters used in the Constructors of Mutex Class in C#.

1. **initiallyOwned:** true to give the calling thread initial ownership of the named system mutex if the named system mutex is created as a result of this call; otherwise, false.
2. **name:** The name of the Mutex. If the value is null, the Mutex is unnamed.
3. **createdNew:** When this method returns, contains a Boolean that is true if a local mutex was created (that is, if the name is null or an empty string) or if the specified named system mutex was created; false if the specified named system mutex already existed. This parameter is passed uninitialized.
4. **mutexSecurity:** A System.Security.AccessControl.MutexSecurity object that represents the access control security to be applied to the named system mutex.

Methods of Mutex Class in C#:

The Mutex Class in C# provides the following methods.

1. **OpenExisting(string name):** This method is used to open the specified named mutex if it already exists. It returns an object that represents the named system mutex. Here, the parameter name specifies the name of the system mutex to open. It will throw ArgumentException if the name is an empty string. -or- name is longer than 260 characters. It will throw ArgumentNullException if the name is null.
2. **OpenExisting(string name, MutexRights rights):** This method is used to open the specified named mutex, if it already exists, with the desired security access. It returns an object that represents the named system mutex. Here, the parameter name specifies the name of the system mutex to open. The parameter rights specify a bitwise combination of the enumeration values that represent the desired security access.
3. **TryOpenExisting(string name, out Mutex result):** This method is used to open the specified named mutex, if it already exists, and returns a value that indicates whether the operation succeeded. Here, the parameter name specifies the name of the system mutex to open. When this method returns, the result contains a Mutex object that represents the named mutex if the call succeeded, or null if the call failed. This parameter is treated as uninitialized. It returns true if the named mutex was opened successfully; otherwise, false.
4. **TryOpenExisting(string name, MutexRights rights, out Mutex result):** This method is used to open the specified named mutex, if it already exists, with the desired security access, and returns a value that indicates whether the operation succeeded. Here, the parameter name specifies the name of the system mutex to open. The parameter rights specify a bitwise combination of the enumeration values that represent the desired security access. When this method returns, the result contains a Mutex object that represents the named mutex if the call succeeded, or null if the call failed. This parameter is treated as uninitialized. It returns true if the named mutex was opened successfully; otherwise, false.
5. **ReleaseMutex():** This method is used to release Mutex once.

6. **GetAccessControl():** This method gets a System.Security.AccessControl.MutexSecurity object that represents the access control security for the named mutex. It returns a System.Security.AccessControl.MutexSecurity object that represents the access control security for the named mutex.
7. **SetAccessControl(mutexSecurity):** This method is used to set the access control security for a named system mutex. The parameter mutexSecurity specifies a System.Security.AccessControl.MutexSecurity object that represents the access control security to be applied to the named system mutex.

Note: The **Mutex** Class in C# is inherited from **WaitHandle** class and the **WaitHandle** class provides the **WaitOne()** method which we need to call to lock the resource. Note that a Mutex can only be released from the same thread which obtained it.

Example to Understand Mutex in C# to Protect Shared Resource in Multithreading:

The following example shows how a local Mutex object is used to synchronize access to a protected resource. Because each calling thread is blocked until it acquires ownership of the mutex, it must call the ReleaseMutex method to release ownership of the mutex. The code is self-explained. So, please go through the comment lines.

```
using System;
using System.Threading;
namespace MutexDemo
{
    class Program
    {
        private static Mutex mutex = new Mutex();
        static void Main(string[] args)
        {
            //Create multiple threads to understand Mutex
            for (int i = 1; i <= 5; i++)
            {
                Thread threadObject = new Thread(MutexDemo)
                {
                    Name = "Thread " + i
                };
                threadObject.Start();
            }
            Console.ReadKey();
        }
        //Method to implement synchronization using Mutex
        static void MutexDemo()
        {
            Console.WriteLine(Thread.CurrentThread.Name + " Wants to Enter Critical
Section for processing");
            try
            {
                //Blocks the current thread until the current WaitOne method returns
                signal.
                //Wait until it is safe to enter.
            }
        }
    }
}
```

```

        mutex.WaitOne();
        Console.WriteLine("Success: " + Thread.CurrentThread.Name + "
Processing now");
        Thread.Sleep(2000);
        Console.WriteLine("Exit: " + Thread.CurrentThread.Name + " is
its task");
    }
    finally
    {
        //Call the ReleaseMutex method to unblock so that other thread
//that are trying to gain ownership of the mutex can enter
        mutex.ReleaseMutex();
    }
}
}
}

```

Output:

```

Thread 1 Wants to Enter Critical Section for processing
Success: Thread 1 is Processing now
Thread 3 Wants to Enter Critical Section for processing
Thread 2 Wants to Enter Critical Section for processing
Thread 4 Wants to Enter Critical Section for processing
Thread 5 Wants to Enter Critical Section for processing
Exit: Thread 1 is Completed its task
Success: Thread 3 is Processing now
Exit: Thread 3 is Completed its task
Success: Thread 5 is Processing now
Exit: Thread 5 is Completed its task
Success: Thread 4 is Processing now
Exit: Thread 4 is Completed its task
Success: Thread 2 is Processing now
Exit: Thread 2 is Completed its task

```

Note: The Mutex Class is inherited from WaitHandle abstract class and the WaitHandle abstract class implements the IDisposable interface. So, when you have finished using the type (in this case Mutex class), you should dispose of it either directly or indirectly. To dispose of the type directly, call its Dispose method in a try/catch/finally block, even though you can also call it from the destructor. To dispose of it indirectly, use a language construct such as using block in C#.

In the following example, each thread calls the WaitOne(Int32) method to acquire the mutex. If the time-out interval elapses, the method returns false, and the thread neither acquires the mutex nor gains access to the resource. The ReleaseMutex method is called only by the thread that acquires the mutex. In the below example, we are calling the Dispose method from the destructor.

```

using System;
using System.Threading;
namespace MutexDemo

```

```

{
    class Program
    {
        private static Mutex mutex = new Mutex();
        static void Main(string[] args)
        {
            //Create multiple threads to understand Mutex
            for (int i = 1; i <= 3; i++)
            {
                Thread threadObject = new Thread(MutexDemo)
                {
                    Name = "Thread " + i
                };
                threadObject.Start();
            }
            Console.ReadKey();
        }
        //Method to implement synchronization using Mutex
        static void MutexDemo()
        {
            // Wait until it is safe to enter, and do not enter if the request
            Console.WriteLine(Thread.CurrentThread.Name + " Wants to Enter Cri
Section for processing");
            if (mutex.WaitOne(1000))
            {
                try
                {
                    Console.WriteLine("Success: " + Thread.CurrentThread.Name
Processing now");
                    Thread.Sleep(2000);
                    Console.WriteLine("Exit: " + Thread.CurrentThread.Name + '
Completed its task");
                }
                finally
                {
                    //Call the ReleaseMutex method to unblock so that other th
                    //that are trying to gain ownership of the mutex can enter
                    mutex.ReleaseMutex();
                    Console.WriteLine(Thread.CurrentThread.Name + " Has Releas
mutex");
                }
            }
            else
            {
                Console.WriteLine(Thread.CurrentThread.Name + " will not acqui
mutex");
            }
        }
    }
    ~Program()
    {
        mutex.Dispose();
    }
}

```

```

    }
}
}

```

Output:

```

Thread 3 Wants to Enter Critical Section for processing
Success: Thread 3 is Processing now
Thread 1 Wants to Enter Critical Section for processing
Thread 2 Wants to Enter Critical Section for processing
Thread 1 will not acquire the mutex
Thread 2 will not acquire the mutex
Exit: Thread 3 is Completed its task
Thread 3 Has Released the mutex

```

OpenExisting Method Example of Mutex Class in C#:

In the below example, we use the OpenExisting method of the Mutex Class in C#. If this method throws an exception, the specified named Mutex does not exist or is inaccessible. The IsSingleInstance method uses this behavior. First, create a console application and copy and paste the following code.

```

using System;
using System.Threading;
namespace MutexDemo
{
    class Program
    {
        static Mutex _mutex;

        static void Main()
        {
            //If IsSingleInstance returns true continue with the Program else
            if (!IsSingleInstance())
            {
                Console.WriteLine("More than one instance"); // Exit program.
            }
            else
            {
                Console.WriteLine("One instance"); // Continue with program.
            }
            // Stay Open.
            Console.ReadLine();
        }
        static bool IsSingleInstance()
        {
            try
            {

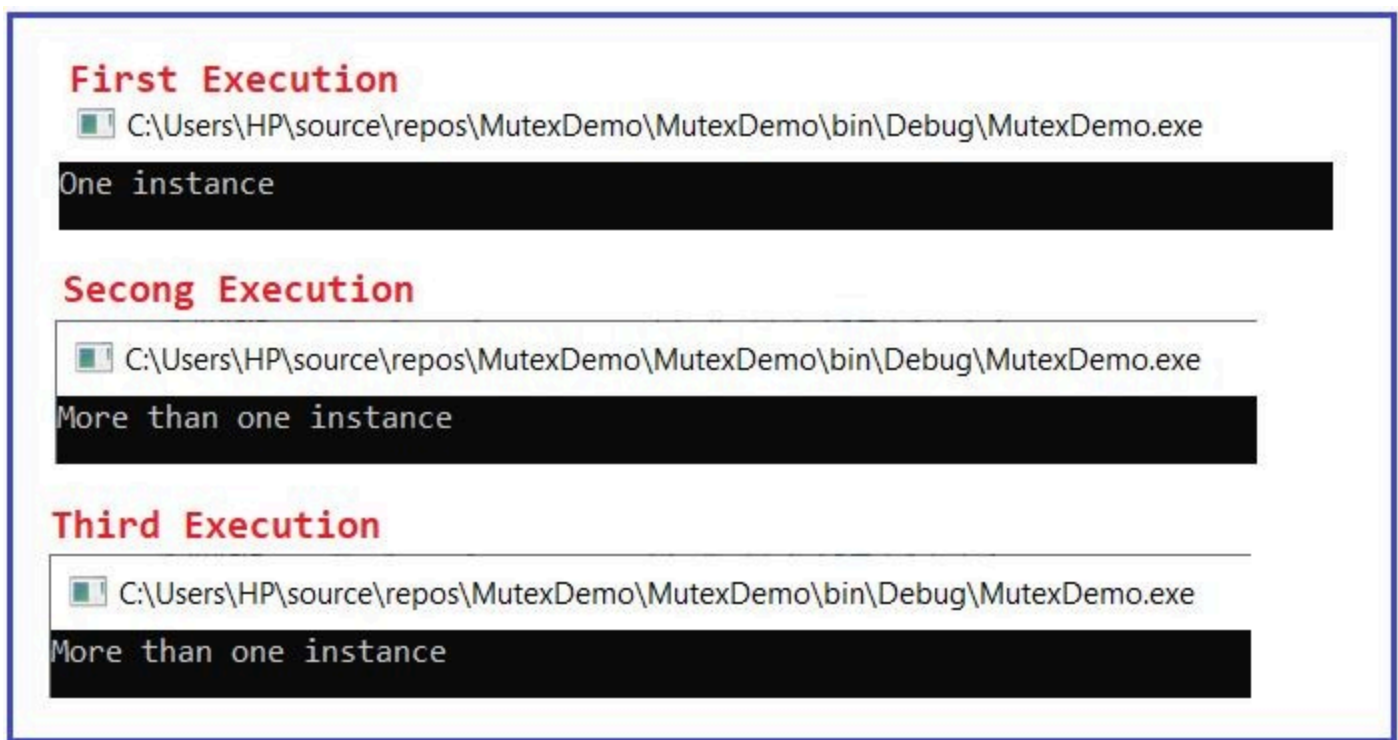
```

```

        // Try to open Existing Mutex.
        //If MyMutex is not opened, then it will throw an exception
        Mutex.OpenExisting("MyMutex");
    }
    catch
    {
        // If exception occurred, there is no such mutex.
        _mutex = new Mutex(true, "MyMutex");
        // Only one instance.
        return true;
    }
    // More than one instance.
    return false;
}
}
}

```

Now, build the project and then go to the projects bin\Debug directory and click on the Application EXE file three times and you will get the following results.



Note: Mutex only allows one external thread to access our application code. But if we want more control over the number of external threads to access our application code, then we need to use Semaphore Class in C#.

In the next article, I am going to discuss the use of [Semaphore Class in C#](#) with Examples. Here, in this article, I try to explain **How to use Mutex in C# to Protect Shared Resources in Multithreading** with Examples. I hope you enjoy this Mutex Class in C# with Examples article.